

Vue globale du cours sur 36 heures

Répartition horaire globale

Module	Thématique	Durée
M0	Environnement, révision POO appliquée au backend	3 h
M1	FastAPI, routing, Pydantic, documentation OpenAPI	6 h
M2	Structuration projet, erreurs, logging, debugging	3 h
M3	Base de données PostgreSQL	6 h
M4	Authentification JWT, autorisation, sécurité API	6 h
M5	Programmation asynchrone avec async/await	3 h
M6	Scripting Python pour la cybersécurité	6 h
M7	Interaction avec APIs tierces et CLI	6 h
M8	Finalisation, tests, démo, évaluation	3 h
Total		36 h

Tableau synthétique des séances

Séance	Durée	Thème principal	Lien cybersécurité
1	3 h	POO, environnement, bases Python	Modélisation d'alertes, actifs, vulnérabilités
2	3 h	FastAPI, routing, Pydantic	API d'alertes avec validation sécurité
3	3 h	Structuration, erreurs, logging	Journalisation d'événements et audit
4	3 h	PostgreSQL, SQLAlchemy	Persistance des actifs et alertes
5	3 h	Relations, migrations, tests	Jeu de données SOC, tests de validation
6	3 h	Authentification JWT	Login sécurisé, tokens, hash de mots de passe
7	3 h	Autorisation, rôles, sécurité API	RBAC analyst/admin/viewer, audit
8	3 h	Async/await	Enrichissement concurrent d'IOC
9	3 h	Scripting, parsing	Analyse de logs SSH/firewall/web
10	3 h	CLI argparse	Outils d'import, export, reporting
11	3 h	APIs tierces	Threat intelligence, enrichissement IP/hash
12	3 h	Finalisation, évaluation	Démo complète du SOC API

Version condensée du plan

Module Python Backend & FastAPI — 36 h

Séance 1 — Python, POO et environnement backend

Révision des classes, héritage, polymorphisme. Modélisation d'objets cybersécurité : actifs, alertes, vulnérabilités.

Séance 2 — FastAPI et Pydantic

Création d'une API REST, routing, validation, documentation Swagger.

Séance 3 — Structuration, erreurs et logging

Organisation routes/services/modèles, gestion des erreurs HTTP, logs, debugging.

Séance 4 — PostgreSQL avec SQLAlchemy

Connexion à PostgreSQL, modèles persistants, premiers CRUD.

Séance 5 — Relations, migrations et tests

Relations entre tables, Alembic, données de démonstration, tests pytest.

Séance 6 — Authentification JWT

Inscription, login, hash des mots de passe, génération et validation de tokens.

Séance 7 — Autorisation et sécurité API

Rôles, permissions, audit, bonnes pratiques sécurité API.

Séance 8 — Async/await

Endpoints asynchrones, concurrence, enrichissement parallèle d'indicateurs.

Séance 9 — Scripting et parsing de logs

Analyse de fichiers logs, extraction d'IP, détection de comportements suspects.

Séance 10 — CLI avec argparse

Industrialisation des scripts, sous-commandes, rapport et automatisation.

Séance 11 — APIs tierces avec requests/httpx

Consommation d'APIs externes, authentification, timeouts, enrichissement threat intel.

Séance 12 — Finalisation et évaluation

Tests, documentation, revue de code, démonstration du projet SecureSOC API.

Découpage détaillé en 12 séances de 3 heures

Séance 1 — Introduction, environnement et révision POO orientée backend

Durée : 3 h

Module : M0

Objectifs pédagogiques

- Mettre en place un environnement Python propre.
- Réviser la POO utile au backend.
- Introduire les bonnes pratiques : venv, git, typage, structure de projet.
- Modéliser des objets métier liés à la cybersécurité.

Contenus

- Présentation du module, des objectifs, du projet fil rouge.
- Rappels Python :
 - classes ;
 - attributs et méthodes ;
 - héritage ;
 - polymorphisme ;
 - dataclasses ;
 - type hints.
- Environnement :
 - création d'un virtualenv ;
 - installation de FastAPI, uvicorn, pydantic ;
 - structure minimale d'un projet ;
 - utilisation de git.

TP cybersécurité

Créer un petit module Python sans API pour modéliser des objets sécurité :

```
class Asset:
    ...

class Server(Asset):
    ...

class NetworkDevice(Asset):
    ...

class Alert:
    ...

class Vulnerability:
    ...
```

Exemple d'exercice :

- créer une classe mère `SecurityEvent` ;
- créer des classes filles : `AuthenticationEvent`, `NetworkEvent`, `MalwareEvent` ;
- implémenter une méthode `to_dict()` polymorphe ;
- ajouter des méthodes de classification de criticité : `low`, `medium`, `high`, `critical`.

Livrables

- Projet Python initialisé.
 - Module `domain/` ou `models/` avec premières classes.
 - Fichier `.gitignore`, `requirements.txt` ou `pyproject.toml`.
-

Séance 2 — Premiers pas avec FastAPI : routes, requêtes, réponses et Pydantic

Durée : 3 h

Module : M1

Objectifs pédagogiques

- Créer une première API FastAPI.
- Comprendre le rôle du routing.
- Utiliser Pydantic pour valider les données.
- Découvrir la documentation OpenAPI/Swagger automatique.

Contenus

- Installation et lancement d'une API FastAPI.
- Routes simples :
 - GET
 - POST
 - PUT
 - DELETE
- Paramètres de chemin :
 - `/assets/{asset_id}`
- Paramètres de requête :
 - `?status=open`
 - `?severity=high`
- Modèles Pydantic :
 - schémas d'entrée ;
 - schémas de sortie ;
 - validation automatique ;

- valeurs par défaut ;
 - champs optionnels.
- Codes HTTP :
 - 200, 201, 204, 400, 404, 422.
- Documentation automatique :
 - /docs
 - /redoc
 - /openapi.json

TP cybersécurité

Créer une API en mémoire, sans base de données, pour gérer des alertes de sécurité.

Endpoints attendus :

```
GET    /health
GET    /alerts
POST   /alerts
GET    /alerts/{alert_id}
PUT    /alerts/{alert_id}
DELETE /alerts/{alert_id}
```

Exemple de modèle Pydantic :

```
class AlertCreate(BaseModel):
    title: str
    description: str
    severity: Literal["low", "medium", "high", "critical"]
    source: str
```

Contraintes de validation à introduire :

- adresse IP valide pour un champ `source_ip` ;
- sévérité limitée à certaines valeurs ;
- longueur maximale du titre ;
- champ `cve_id` optionnel avec format simple.

Livrables

- API FastAPI fonctionnelle.
 - Documentation Swagger accessible.
 - Jeu d'essai manuel via Swagger ou curl.
-

Séance 3 — Structuration du projet, gestion des erreurs HTTP et debugging

Durée : 3 h

Module : M2

Objectifs pédagogiques

- Structurer un projet backend professionnellement.
- Séparer les responsabilités : routes, services, schémas, configuration.
- Gérer proprement les erreurs HTTP.
- Mettre en place logging et debugging.

Contenus

Structure de projet proposée :

```
securesoc-api/  
  app/  
    main.py  
    core/  
      config.py  
      logging.py  
      exceptions.py  
    api/  
      routes/  
        alerts.py  
        assets.py  
        health.py  
    services/  
      alert_service.py  
      asset_service.py  
    schemas/
```

```
    alert.py
    asset.py
    repositories/
        memory_repository.py
tests/
requirements.txt
README.md
```

Notions abordées :

- séparation routes / services / modèles ;
- configuration avec variables d'environnement ;
- exceptions personnalisées ;
- handlers globaux d'erreurs ;
- réponses d'erreur structurées ;
- logging structuré ;
- debugging avec breakpoints ;
- utilisation de `uvicorn --reload`.

Exemple de réponse d'erreur structurée

```
{
  "error": "ALERT_NOT_FOUND",
  "message": "Alert with id 42 not found",
  "status_code": 404
}
```

TP cybersécurité

Améliorer l'API de la séance précédente :

- déplacer la logique métier dans un service ;
- créer un gestionnaire d'erreurs pour les alertes introuvables ;
- logger les créations et suppressions d'alertes ;
- ajouter un middleware ou handler pour journaliser les erreurs ;
- simuler des erreurs et observer les logs.

Exemple de log attendu :


```
2026-08-28 10:15:23 WARNING app.services.alert_service Alert 42 not found
```

Livrables

- Projet restructuré.
 - Gestion d'erreurs propre.
 - Logs applicatifs.
 - Premier fichier README.md.
-

Séance 4 — Connexion à PostgreSQL avec SQLAlchemy

Durée : 3 h

Module : M3

Objectifs pédagogiques

- Comprendre la persistance des données.
- Connecter FastAPI à PostgreSQL.
- Créer des modèles de base de données.
- Mettre en place un premier CRUD persistant.

Contenus

- Introduction à PostgreSQL.
- Démarrage d'une base via Docker Compose ou serveur local.
- SQLAlchemy :
 - moteur ;
 - session ;
 - modèles ;
 - déclarations de tables.
- Intégration avec FastAPI :
 - dépendance `get_db` ;
 - cycle de vie d'une session ;
 - injection de dépendances.

Modèle de base possible

```
users
assets
alerts
vulnerabilities
audit_logs
```

Exemple de tables simples :

```
assets
- id
- hostname
- ip_address
- asset_type
- criticality
- created_at

alerts
- id
- title
- description
- severity
- status
- asset_id
- created_at
```

TP cybersécurité

Mettre en place la persistance pour les entités :

- Asset
- Alert

Fonctionnalités attendues :

- créer un actif ;
- lister les actifs ;
- récupérer un actif par id ;
- créer une alerte liée à un actif ;
- lister les alertes avec filtres :

- par sévérité ;
- par statut ;
- par actif.

Livrables

- Base PostgreSQL fonctionnelle.
 - Modèles SQLAlchemy.
 - Endpoints persistants pour assets et alerts.
-

Séance 5 — Relations, migrations, données de test et introduction aux tests

Durée : 3 h

Module : M3

Objectifs pédagogiques

- Gérer les relations entre tables.
- Mettre en place des migrations avec Alembic.
- Créer des données de démonstration.
- Introduire les tests automatisés.

Contenus

- Relations SQLAlchemy :
 - un actif peut avoir plusieurs alertes ;
 - une vulnérabilité peut concerner plusieurs actifs ;
 - un utilisateur peut créer ou traiter des alertes.
- Alembic :
 - initialisation ;
 - génération de migration ;
 - application de migration.
- Seed :
 - insertion d'utilisateurs ;
 - insertion d'actifs ;

- insertion d’alertes ;
- insertion de vulnérabilités.
- Tests :
 - pytest ;
 - client de test FastAPI ;
 - tests de routes ;
 - tests de validation.

Exemples de tests

```
- POST /alerts avec données valides => 201
- POST /alerts avec sévérité invalide => 422
- GET /alerts/{id} inconnu => 404
- GET /assets?criticality=high => 200
```

TP cybersécurité

Créer un jeu de données type SOC :

```
- 3 utilisateurs
- 10 actifs
- 20 alertes
- 10 vulnérabilités
```

Exemples de données :

```
Actif : web-server-01
IP : 10.10.10.15
Type : server
Criticality : high

Alerte : Tentative de connexion SSH suspecte
Severity : high
Status : open

Vulnérabilité : CVE-2024-XXXX
Severity : critical
```

Ajouter des tests sur :

- création d'actif ;
- création d'alerte ;
- validation d'adresse IP ;
- filtrage par criticité.

Livrables

- Migrations Alembic.
 - Script de seed.
 - Premiers tests automatisés.
-

Séance 6 — Authentification : utilisateurs, mots de passe et JWT

Durée : 3 h

Module : M4

Objectifs pédagogiques

- Créer un système d'authentification.
- Hasher les mots de passe.
- Générer et vérifier des tokens JWT.
- Sécuriser les endpoints.

Contenus

- Différence authentification / autorisation.
- Bonnes pratiques mots de passe :
 - jamais en clair ;
 - hash robuste ;
 - utilisation de bibliothèques adaptées.
- Flux d'authentification :
 - création de compte ;
 - login ;
 - obtention d'un token ;
 - accès à une ressource protégée.
- JWT :

- header ;
 - payload ;
 - signature ;
 - expiration ;
 - claims.
- FastAPI :
 - dépendance `get_current_user` ;
 - schéma `OAuth2PasswordRequestForm` ou équivalent ;
 - protection de routes.

Endpoints à créer

```
POST /auth/register
POST /auth/login
GET /auth/me
```

Exemple de payload JWT

```
{
  "sub": "user-id",
  "email": "analyst@soc.local",
  "role": "analyst",
  "exp": 1730000000
}
```

TP cybersécurité

Implémenter l'authentification pour l'API SOC :

- création d'utilisateurs ;
- rôle stocké : **admin**, **analyst**, **viewer** ;
- login retourne un JWT ;
- les routes sensibles nécessitent un token ;
- les mots de passe sont hashés.

Exemples de règles :

- `/auth/register` public ou réservé admin selon choix pédagogique ;

- `/assets` accessible en lecture aux utilisateurs authentifiés ;
- `/alerts` modifiable seulement par `analyst` ou `admin`.

Livrables

- Système de login.
 - JWT fonctionnel.
 - Routes protégées.
 - Démonstration d'accès refusé sans token.
-

Séance 7 — Autorisation, rôles, sécurité API et bonnes pratiques

Durée : 3 h

Module : M4

Objectifs pédagogiques

- Mettre en place une autorisation par rôles.
- Protéger les endpoints selon les permissions.
- Sensibiliser aux risques de sécurité API.
- Ajouter des journaux d'audit.

Contenus

- Gestion des rôles :
 - `admin`
 - `analyst`
 - `viewer`
- Dépendances FastAPI :
 - `get_current_user`
 - `require_admin`
 - `require_analyst`
- Autorisation fine :
 - lecture seule ;
 - création/modification ;

- suppression réservée admin.
- Bonnes pratiques sécurité :
 - ne pas exposer d'informations sensibles ;
 - valider toutes les entrées ;
 - gérer les secrets via variables d'environnement ;
 - limiter les informations dans les erreurs ;
 - journaliser les actions sensibles ;
 - activer HTTPS en production ;
 - configurer CORS strictement.
- Introduction à OWASP API Security Top 10 :
 - broken object level authorization ;
 - broken authentication ;
 - excessive data exposure ;
 - security misconfiguration ;
 - improper assets management.

TP cybersécurité

Mettre en place une matrice d'autorisation.

Exemple :

Action	Viewer	Analyst	Admin
Lister assets	Oui	Oui	Oui
Créer asset	Non	Oui	Oui
Modifier asset	Non	Oui	Oui
Supprimer asset	Non	Non	Oui
Lister alerts	Oui	Oui	Oui
Créer alert	Non	Oui	Oui
Fermer alert	Non	Oui	Oui
Créer utilisateur	Non	Non	Oui

Exercices :

- créer un utilisateur **viewer** ;
- vérifier qu'il ne peut pas créer d'alerte ;
- vérifier qu'un **analyst** ne peut pas supprimer un actif ;
- journaliser les actions sensibles :
 - login ;

- création d’alerte ;
- suppression d’actif ;
- modification de statut d’alerte.

Livrables

- Autorisation par rôles.
 - Journal d’audit.
 - Tests d’accès non autorisés.
 - Documentation des permissions dans le README.
-

Séance 8 — Programmation asynchrone : `async/await` et concurrence

Durée : 3 h

Module : M5

Objectifs pédagogiques

- Comprendre l’intérêt de l’asynchrone pour un backend.
- Utiliser `async/await` dans FastAPI.
- Gérer des appels concurrents.
- Mesurer l’impact sur les performances.

Contenus

- Différence entre code synchrone et asynchrone.
- Cas d’usage :
 - appels réseau ;
 - accès base de données ;
 - requêtes vers APIs tierces.
- Fonctions asynchrones :
 - `async def`
 - `await`
 - `asyncio.gather`
- FastAPI asynchrone :

- endpoints `async def`
- dépendances asynchrones
- Bonnes pratiques :
 - timeouts ;
 - gestion d'erreurs ;
 - éviter les opérations bloquantes dans une fonction asynchrone ;
 - utiliser un client HTTP asynchrone si nécessaire.

TP cybersécurité

Créer un endpoint d'enrichissement d'indicateurs.

Exemple :

```
GET /enrich/ip/{ip}
```

Objectif :

- interroger plusieurs sources simulées :
 - réputation IP ;
 - géolocalisation ;
 - liste de blocage ;
 - historique d'alertes internes.
- exécuter les appels en parallèle avec `asyncio.gather`.

Exemple de réponse :

```
{
  "ip": "192.0.2.10",
  "reputation_score": 78,
  "is_blocked": true,
  "country": "Unknown",
  "alerts_count": 3,
  "sources": [
    "internal",
    "blocklist",
    "reputation"
  ]
}
```

Variante sans vraie API externe :

- créer des fonctions asynchrones simulées ;
- ajouter des délais artificiels ;
- comparer le temps total avec et sans parallélisation.

Livrables

- Endpoint asynchrone fonctionnel.
 - Démonstration de concurrence.
 - Gestion des timeouts et erreurs.
-

Séance 9 — Scripting Python : parsing de fichiers et analyse de logs

Durée : 3 h

Module : M6

Objectifs pédagogiques

- Automatiser des tâches courantes avec Python.
- Lire et parser des fichiers CSV, JSON, logs texte.
- Extraire des indicateurs de compromission simples.
- Produire un rapport exploitable.

Contenus

- Lecture de fichiers :
 - `open()`
 - `pathlib`
 - encodage ;
 - fichiers volumineux.
- Formats courants :
 - CSV ;
 - JSON ;
 - logs ligne à ligne.
- Expressions régulières :
 - extraction d'IP ;

- extraction de dates ;
 - extraction de statuts HTTP ;
 - extraction de hash.
- Structures de données :
 - listes ;
 - dictionnaires ;
 - compteurs ;
 - regroupements.
- Écriture de résultats :
 - JSON ;
 - CSV ;
 - Markdown.

TP cybersécurité

Fournir aux étudiants des logs fictifs, par exemple :

```
2026-08-28T08:12:01 sshd Failed password for admin from 203.0.113.10 port 22
2026-08-28T08:12:04 sshd Failed password for admin from 203.0.113.10 port 22
2026-08-28T08:12:07 sshd Failed password for root from 203.0.113.10 port 22
2026-08-28T08:15:33 nginx 200 GET /login 10.20.30.40
2026-08-28T08:16:02 nginx 401 POST /login 198.51.100.23
```

Script attendu :

```
scripts/parse_logs.py
```

Fonctionnalités :

- lire un fichier de logs ;
- détecter les IP ayant plusieurs tentatives de connexion échouées ;
- compter les erreurs HTTP 401 ou 403 ;
- extraire les hashes éventuels ;
- produire un rapport JSON :

```
{
  "suspicious_ips": [
    {
      "ip": "203.0.113.10",
      "failed_attempts": 3
    }
  ],
  "total_events": 120,
  "alerts_generated": 1
}
```

Livrables

- Script de parsing.
 - Rapport JSON ou CSV.
 - Petite documentation d'utilisation.
-

Séance 10 — CLI avec argparse et industrialisation des scripts

Durée : 3 h

Module : M6

Objectifs pédagogiques

- Créer des outils en ligne de commande.
- Utiliser argparse.
- Gérer arguments, options et sous-commandes.
- Rendre un script robuste et réutilisable.

Contenus

- Module `argparse` :
 - arguments positionnels ;
 - options ;
 - valeurs par défaut ;
 - types ;

- aide ;
 - sous-commandes.
- Bonnes pratiques CLI :
 - codes de sortie ;
 - messages d’erreur clairs ;
 - mode verbeux ;
 - configuration par fichier ou variables d’environnement ;
 - logging.
- Automatisation :
 - import de logs ;
 - export de rapport ;
 - création d’utilisateur admin ;
 - purge d’anciennes données.

Exemple de CLI

```
python scripts/securesoc_cli.py parse-logs --input logs/auth.log --output report.json
python scripts/securesoc_cli.py create-admin --email admin@soc.local
python scripts/securesoc_cli.py export-alerts --severity high --format csv
python scripts/securesoc_cli.py enrich-ip --ip 203.0.113.10
```

TP cybersécurité

Transformer les scripts de la séance précédente en CLI complète.

Sous-commandes à implémenter :

```
parse-logs
export-alerts
create-admin
audit-summary
```

Exemple :

```
python scripts/securesoc_cli.py parse-logs \
  --input data/logs/firewall.log \
  --threshold 5 \
  --output reports/incidents.json
```

Le script doit :

- accepter un fichier d'entrée ;
- définir un seuil de détection ;
- produire un rapport ;
- retourner un code de sortie différent si un problème survient.

Livrables

- CLI opérationnelle.
 - Documentation dans le README.
 - Exemples de commandes.
-

Séance 11 — Interaction avec APIs tierces via requests/httpx

Durée : 3 h

Module : M7

Objectifs pédagogiques

- Consommer une API REST externe.
- Réaliser des requêtes GET et POST.
- Gérer authentification, erreurs et timeouts.
- Intégrer l'enrichissement externe dans l'API FastAPI.

Contenus

- Bibliothèque `requests` :
 - GET ;
 - POST ;
 - headers ;
 - JSON ;
 - paramètres ;
 - codes de statut.
- Alternative asynchrone :
 - `httpx`

- intérêt dans une API FastAPI asynchrone.
- Gestion des erreurs :
 - timeout ;
 - API indisponible ;
 - réponse invalide ;
 - retry simple.
- Authentification externe :
 - API key dans header ;
 - ne jamais stocker la clé en dur ;
 - variables d’environnement ;
 - fichier `.env` non versionné.

TP cybersécurité

Créer un service d’enrichissement d’IOC.

Cas d’usage :

- enrichir une IP ;
- enrichir un hash ;
- enrichir un domaine.

Si aucune vraie API externe n’est disponible, créer une fausse API locale ou utiliser des fixtures JSON.

Exemple de service :

```
class ThreatIntelClient:
    def get_ip_reputation(self, ip: str) -> dict:
        ...

    def get_hash_report(self, file_hash: str) -> dict:
        ...
```

Endpoint FastAPI associé :

```
GET /enrich/ip/{ip}
GET /enrich/hash/{hash}
```

Exemple de réponse :


```
{
  "indicator": "203.0.113.10",
  "type": "ip",
  "risk_score": 82,
  "tags": [
    "brute-force",
    "malicious"
  ],
  "first_seen": "2026-08-01",
  "last_seen": "2026-08-27"
}
```

Consignes de sécurité :

- ne pas exposer la clé API dans la réponse ;
- journaliser l'appel sans loguer le secret ;
- gérer proprement l'échec de l'API tierce.

Livrables

- Client API tierce.
 - Endpoint d'enrichissement.
 - Gestion des erreurs réseau.
 - Configuration par variables d'environnement.
-

Séance 12 — Finalisation, tests, revue de code et soutenance

Durée : 3 h

Module : M8

Objectifs pédagogiques

- Finaliser le projet.
- Vérifier la qualité globale du code.
- Tester et démontrer le fonctionnement.
- Présenter les choix techniques et les limites.

Activités

- Finalisation du projet fil rouge :
 - correction des bugs ;
 - amélioration de la documentation ;
 - vérification des tests ;
 - nettoyage du code.
- Revue de code :
 - nommage ;
 - structure ;
 - gestion d'erreurs ;
 - sécurité ;
 - lisibilité.
- Démonstration :
 - démarrage de l'API ;
 - Swagger ;
 - création d'utilisateur ;
 - login ;
 - création d'actif ;
 - création d'alerte ;
 - restriction par rôle ;
 - parsing de logs ;
 - enrichissement d'IP.
- Évaluation ou mini-soutenance.

Livrables finaux

- Dépôt Git propre.
 - README complet.
 - API fonctionnelle.
 - Scripts CLI.
 - Jeu de données de démonstration.
 - Tests automatisés.
 - Documentation OpenAPI.
-
-